

Публикация контента в каналах МАХ

Техническое описание текущей реализации
проект max-repost

Документ описывает, как устроены два независимых контура публикации:

- 1) обычный репост из внешних источников (Telegram, VK, RSS, X);
- 2) SEO-маяки футбольных матчей (смена названия канала + промо-пост).

Корень проекта: /var/max-repost

База данных: /var/max-repost/data/app.db

Дата документа: август 2026

1. Общая архитектура

Система состоит из трёх systemd-сервисов, общей SQLite-базы и клиента MAX Platform API. Репост и SEO-маяки работают независимо, но используют один и тот же MaxClient и одни и те же каналы MAX.

1.1. Сервисы

- max-repost-admin — веб-админка (FastAPI / uvicorn), файл app/main.py, скрипт scripts/run-admin.sh.
- max-repost-worker — цикл синхронизации чатов, опроса источников и публикации репостов: workers/run.py.
- max-repost-seo — отдельный воркер SEO-маяков: workers/seo_run.py → seo/cycle.py.

1.2. Роли компонентов

- Админка: логин, список каналов, watermark/footer, источники, история постов, страница SEO и принудительная ротация (/seo/force/{slug}).
- Worker репоста: sync_chats → poll_sources → publish_pending в бесконечном цикле main_loop.
- SEO worker: run_seo_tick каждые SEO_POLL_INTERVAL_SEC (по умолчанию 300 с).
- Хранилище: SQLite в режиме WAL (Settings.db_path → data/app.db), доступ через app/db.py.

1.3. Клиент MAX API

Класс MaxClient в app/max_api.py. База: MAX_API_BASE (по умолчанию https://platform-api2.max.ru), авторизация заголовком с токеном бота. Повторы при HTTP 429 — до 5 попыток в _request.

Основные методы:

- list_chats / get_chat / get_updates — синхронизация каналов и событий bot_added / bot_removed / bot_stopped.
- send_message — публикация поста (POST /messages?chat_id=...).
- upload_image_* / upload_video_* — двухшаговая загрузка через POST /uploads?type=image|video и upload URL.
- delete_message, pin_message, patch_chat — используются в SEO (удаление старого поста, закреп, смена title).

2. Обычный репост (контент из источников)

Это основной контур: посты из Telegram / VK / RSS / X проходят фильтр, попадают в очередь posts и публикуются в целевой канал MAX.

2.1. Схема потока

```
sources (SQLite)
→ poll_sources() [workers/run.py]
→ parsers (tg|vk|rss|x)
→ is_advertisement / require_media
→ insert_post (+ hold publish_at)
→ claim_pending_posts
→ sanitize + upload media + send_message
```

→ register_send / mark sent|error|skipped

2.2. Источники и парсеры

- Типы sources.kind: telegram | vk | rss | x.
- Парсеры: parsers/telegram.py, parsers/vk.py, parsers/rss.py, parsers/x.py — общий формат ParsedPost.
- Определение типа: detect_kind + добавление через админку (add_source_route).
- Первый опрос (bootstrap): история не публикуется; выставляется sources.cursor как watermark курсора.

2.3. Модерация и правила

Правила загружаются из YAML: rules/core.yaml и оверрайды rules/channels/*.yaml (app/rules.py → resolve_rules, sync_rules_to_db при init_db).

- moderation_mode=strip (ядро): жёсткий блок рекламы/стримов/капперов (is_advertisement); при публикации sanitize_for_publish вырезает ссылки и СТА.
- moderation_mode=strict: дополнительно блокирует любой URL в тексте; без strip-вырезки ссылок.
- require_media: если true — пост без медиа не берётся в публикацию (has_publishable_media).
- Примеры оверрайдов: goroskop.yaml (require_media: false), igry.yaml (strip + require_media).

2.4. Per-channel настройки в БД

- watermark_text — текст на картинках (app/watermark.py, Inter, верхний правый угол).
- footer_link / footer_link_text / footer_as_button — хвост поста: HTML-ссылка или inline_keyboard.
- moderation_mode, require_media — синхронизируются из YAML.
- hidden — канал исключается из опроса.

Колонка instant_publish есть в схеме channels, но worker её сейчас не читает.

2.5. Очередь и удержание

- PUBLISH_HOLD_SEC (default 180): при insert_post выставляется publish_at = now + hold, чтобы дать время на правки/дубли.
- REPUBLISH_WINDOW_SEC (default 600): похожие pending того же источника помечаются skipped (supersede_similar_pending + texts_similar).
- claim_pending_posts переводит pending → queued перед отправкой.

2.6. Публикация (publish_pending)

Для каждого claimed-поста worker:

- при необходимости переводит текст (translate_to_russian), если sources.translate=1;
- прогоняет sanitize_for_publish / scrub_promo_media;
- загружает медиа (изображения с опциональным watermark; видео без watermark);
- для больших TG-медиа использует Telethon (app/tg_mtproto.py → data/tg_cache/);
- регистрирует отправку в send_log (антидубль по external_id и content_hash);
- вызывает MaxClient.send_message с attachments и опциональной кнопкой footer.

2.7. Обработка ошибок

- Сбой upload медиа → удаление записи send_log, статус skipped.
- MaxApiError attachment.not.ready → снова pending (повтор на следующем тике).
- Прочий MaxApiError → status=error.
- Сбой перевода / TG download → pending (+ для TG retry через ~120 с).
- HTTP 429 на стороне MAX — retry внутри MaxClient._request.

Отдельной экспоненциальной очереди retry нет: повтор опирается на следующий тик PUBLISH_INTERVAL_SEC и статусы posts.

3. Медиа

3.1. Изображения

- Репост: upload_image_from_url → опционально apply_text_watermark → upload_image_bytes.
- SEO: байты generate_cover_image → upload_image_bytes (watermark канала не применяется).

3.2. Видео

- upload_video_from_url / upload_video_from_path / upload_video_bytes.
- Лимит порядка ~250 MB на стороне клиента.
- Водермарка на видео не ставится.

3.3. Telegram «Media is too big»

Парсер помечает too_big + tg_ref. При публикации Telethon скачивает файл в data/tg_cache/ (сессия data/tg_user.session, логин scripts/tg-login.py). Если скачать не удалось — пост остаётся pending с отложенным publish_at.

Пакет yt-dlp указан в requirements.txt, но в текущем коде публикации (app/workers/parsers) не вызывается.

4. SEO-маяки (отдельный контур)

SEO-каналы не репостят чужой контент. Они выбирают «громкий» ближайший матч, переименовывают канал под SEO-заголовок и публикуют один закреплённый промо-пост со ссылкой на основное сообщество.

4.1. Конфиг

YAML в seo/channels/*.yaml, модель SeoChannelConfig (seo/channel_config.py). Активные маяки: aplmatch, champions_lige, fifamatch, italymatch, laligamatch, uefafootball.

Ключевые поля: chat_id, competitions, providers, horizon_days, pick_mode (hype | national_priority), hype_window_days / priority_window_days, title_suffix, idle_title, target_community, pin_post, labels, priority_teams.

4.2. Цикл (seo/cycle.py)

```
run_seo_tick / process_channel
    → fetch_matches_for_competitions (football-data + ESPN)
    → pick_top_match | pick_national_priority_match
```

```

→ match_needs_rotation?
→ rotate_channel:
    delete_message (старый mid)
    build_seo_title → patch_chat(title)
    generate_post_text + generate_cover_image
    send_message + кнопка СТА
    pin_message (если pin_post)
    upsert_seo_active
→ нет матчей → apply_idle (idle_title + удаление поста)

```

4.3. Выбор матча (seo/fixtures.py)

Режим hure (клубные лиги: PL, PD, SA, CL, EL)

- Окно хайпа: от ближайшего кикоффа + N дней (обычно 4), не весь месяц.
- Основной ранг: престиж пары $\min(\text{club_a}, \text{club_b}) + 0.5 \times \max$ + стадия турнира.
- Если лидер один в пределах ~5% — берём его без внешних сигналов.
- Иначе тай-брейк: Google Trends (seo/trends.py), затем Yandex Wordstat (seo/wordstat.py).
- При пустом Wordstat / квоте 429 — остаётся лидер по престижу (ИИ больше не перебивает выбор).

Режим national_priority (fifamatch)

- Сначала основные стадии WC/EC/CA в окне — тот же prestige-пайплайн.
- Иначе: окно от ближайшего матча и приоритет сборных Россия → Испания → Франция → Аргентина, затем время.

4.4. Заголовок и пост

- Заголовок: «Клуб1 — Клуб2 матч смотреть онлайн...» (Title Case клубов + em dash, суффикс из title_suffix). Имена: seo/titles.py + seo/team_names_ru.yaml.
- Текст поста: блоки с пустыми строками, клубы с заглавных, СТА на target_community (обычно https://max.ru/channel_vnfutbol).
- Обложка: логотипы клубов + дата (PIL), опциональный AI-фон через openclaw; файлы в data/seo_covers/.

4.5. Отличие от репоста

- Нет источников TG/VK и очереди posts/send_log репоста.
- Меняется title канала MAX; держится один закреплённый промо-пост.
- Состояние хранится в seo_active_matches.
- Принудительная ротация: scripts/seo_smoke.py --live --slug ... или админка /seo/force/{slug}.

5. Антидубли и статусы

- UNIQUE(source_id, external_id) на posts.
- send_log: UNIQUE(chat_id, external_id) и UNIQUE(chat_id, content_hash); fingerprint = SHA-256(chat_id|text|media_json).
- supersede_similar_pending — похожие свежие pending того же источника.

Статусы posts.publish_status: pending → queued → sent | skipped | error (плюс удержание publish_at).

6. Таблицы БД, связанные с публикацией

- channels — каналы MAX: watermark, footer, moderation, require_media, hidden, last_published_at.
- sources — источники, cursor, translate, status.
- posts — очередь контента и статусы публикации.
- send_log — журнал отправок / антидубль.
- meta — служебные ключи (например updates_marker).
- seo_active_matches — текущий матч, mid, title SEO-маяка.

Схема и миграции колонок: app/db.py (SCHEMA + _ensure_column в init_db).

7. Конфигурация (имена переменных)

Секреты в документе не приводятся. Релевантные имена из app/config.py / .env:

- MAX: MAX_BOT_TOKEN, MAX_API_BASE, ADMIN_*
- Интервалы: POLL_INTERVAL_SEC, SYNC_CHATS_INTERVAL_SEC, PUBLISH_INTERVAL_SEC, PUBLISH_HOLD_SEC, REPUBLISH_WINDOW_SEC
- Парсеры/перевод: VK_ACCESS_TOKEN, SCRAPER_HTTP_PROXY, GROQ_*, TRANSLATE_BACKEND, OPENCLAW_*, OPENAI_*
- SEO: FOOTBALL_DATA_TOKEN, SEO_POLL_INTERVAL_SEC, SEO_CHANNELS_DIR, WORDSTAT_*, YANDEX_*
- TG MTPROTO: TG_API_ID, TG_API_HASH, TG_SESSION_PATH

8. Карта ключевых файлов

```
/var/max-repost/
app/main.py           # админка
app/max_api.py        # MaxClient
app/db.py             # SQLite
app/filter.py         # модерация / sanitize
app/rules.py          # YAML-правила
app/watermark.py      # watermark на картинках
app/tg_mtproto.py     # скачивание больших TG-медиа
app/translate.py      # перевод на русский
workers/run.py        # репост-воркер
workers/seo_run.py    # SEO-воркер
parsers/*             # TG / VK / RSS / X
rules/core.yaml + channels/
seo/cycle.py, content.py, fixtures.py, titles.py,
  channel_config.py, wordstat.py, trends.py
seo/channels/*.yaml
data/app.db
```

9. Краткие выводы

- В MAX публикуются два типа контента: репосты из внешних лент и SEO-промо матчей.
- Единая точка отправки — MaxClient.send_message (+ uploads); SEO дополнительно меняет title и pin.
- Качество репоста обеспечивается правилами YAML, фильтром рекламы, hold-окном и send_log.

- Качество SEO — окном 4 дня и ранжированием по престижу пары с осторожным тай-брейком Trends/Wordstat.

Документ отражает состояние реализации на момент составления и предназначен для внутренней документации проекта max-repost.